.NET MAUI
Build Modern Apps with .NET 11



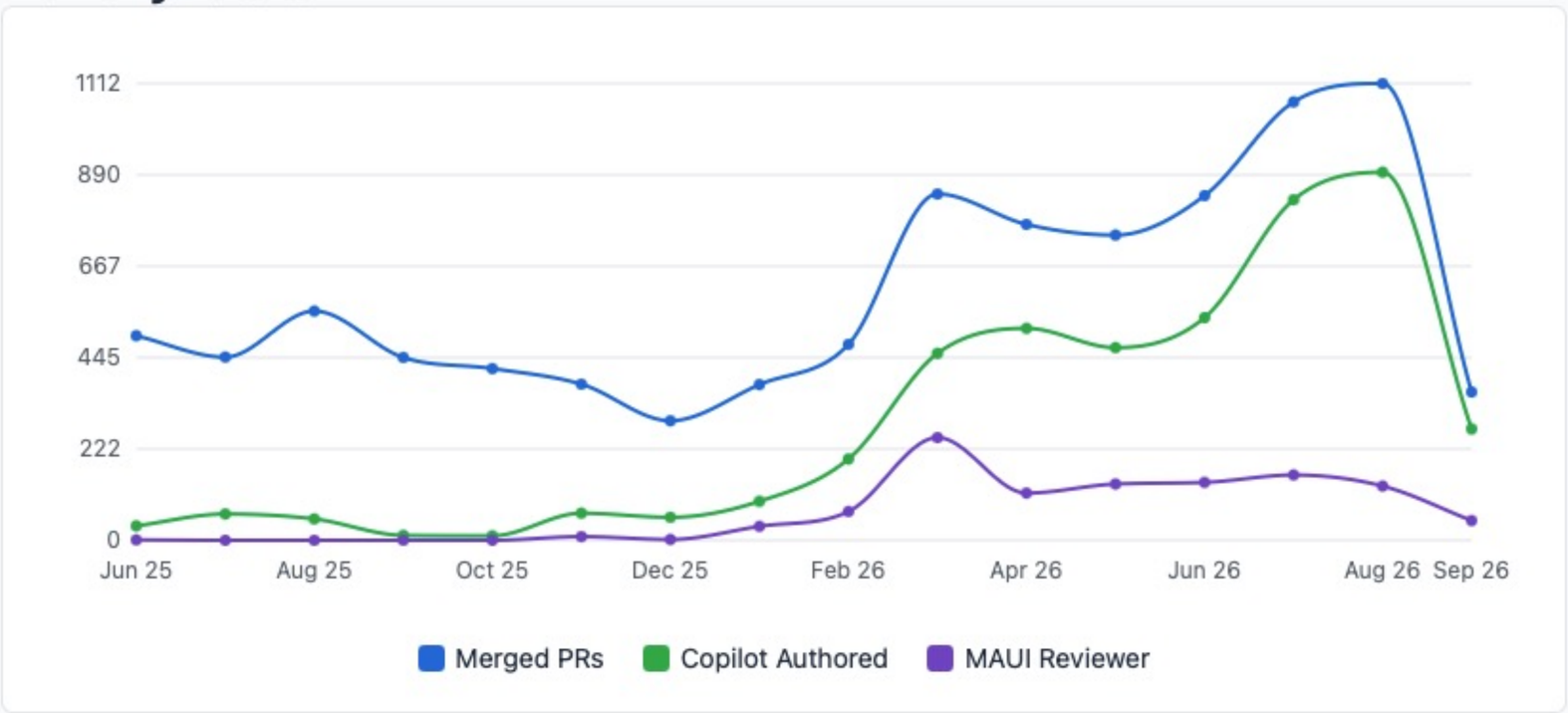
David Ortinau
Principal Product Manager
.NET MAUI

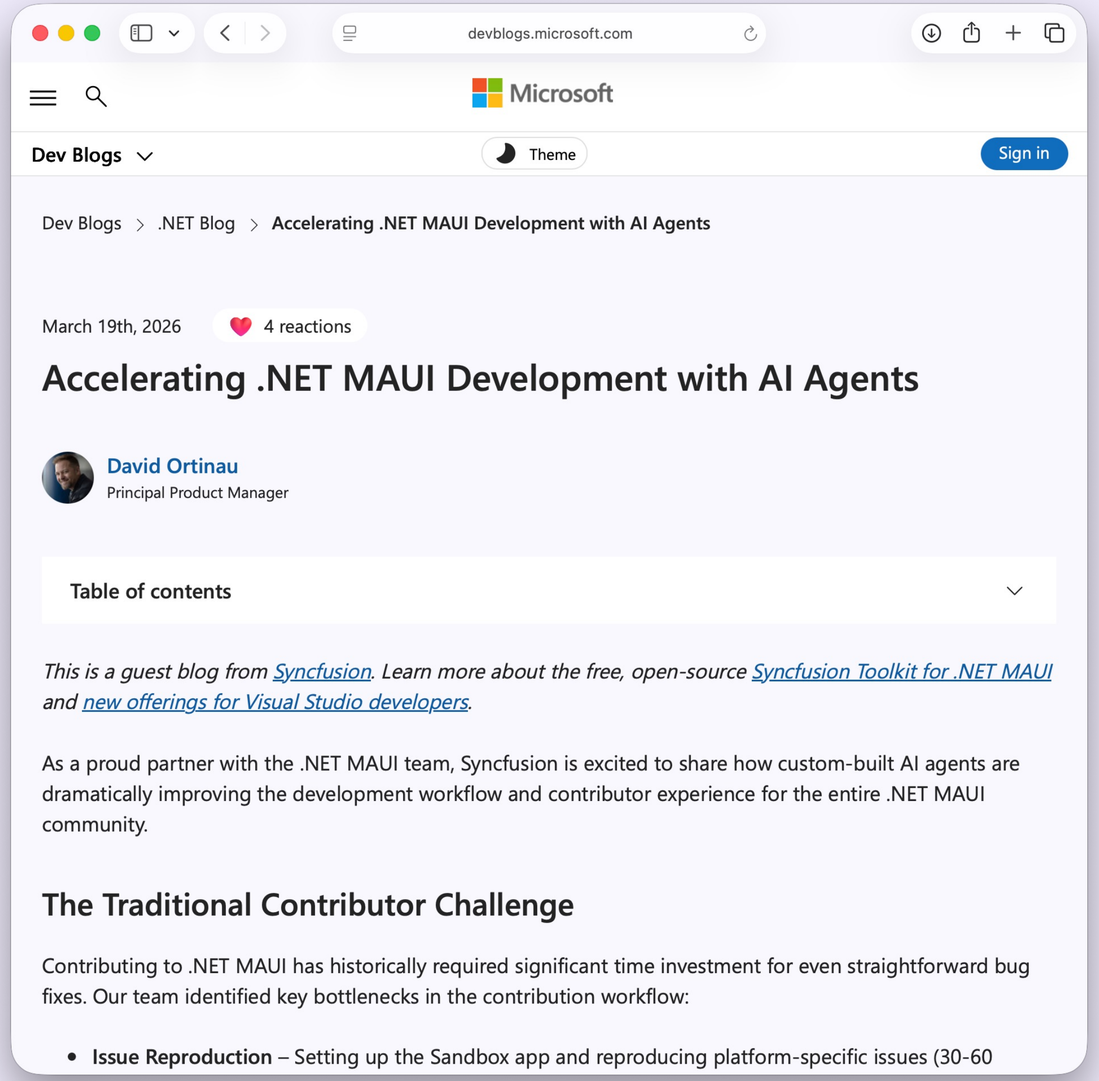
.NET

.NET MAUI
Health Check

Total Merged PRs	Copilot Authored	MAUI Reviewer Used
9626	4567	1098

Monthly Totals





- **50-70% reduction** in time per issue
- **2-3x increase** in issues addressed per week
- **95%+ of PRs** include comprehensive tests (up from ~60%)
- **Lower barrier** to community contribution
- **Higher quality** through multi-model fix exploration

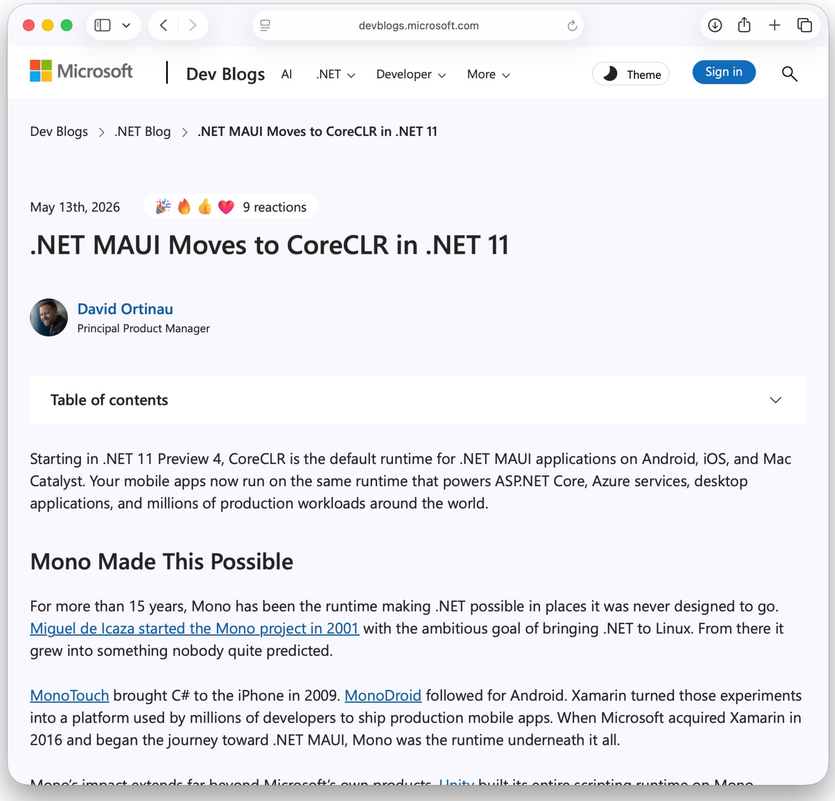
Syncfusion team-reported results

.NET Blog, 19 March 2026

Measurement window and sample size not stated.

Coverage means PRs with tests, not code coverage.

CoreCLR becomes the MAUI default



DEFAULT IN .NET 11

Target .NET 11

CoreCLR is the default on Android, iOS, and Mac Catalyst. No opt-in is needed.

ANDROID RELEASE: OPTIONAL

```
<MauiEnableFullReadyToRun>
  true
</MauiEnableFullReadyToRun>
```

In .csproj / PropertyGroup. Partial ReadyToRun is the default; full ReadyToRun trades package size for startup performance.

NativeAOT is a separate, explicit publish model.

PLATFORMS

Where can you run .NET MAUI apps in 2026?

01

Mobile

Android
iOS

02

Desktop

Linux
macOS
Mac Catalyst
Windows

03

Web

Blazor Hybrid
Hybrid

04

TV

Apple TV, Google TV, Samsung Tizen

05

Auto

Apple CarPlay, Android Auto

RELEASE CONTEXT

Let's explore 2 amazing releases

MAUI 11

Gestures and swipe colors

Routes, badges, maps and passkeys

XAML, runtime and platform appearance

Toolkit 1.0.11

Grid Splitter

Interactive Viewer

Chart tooltip styling

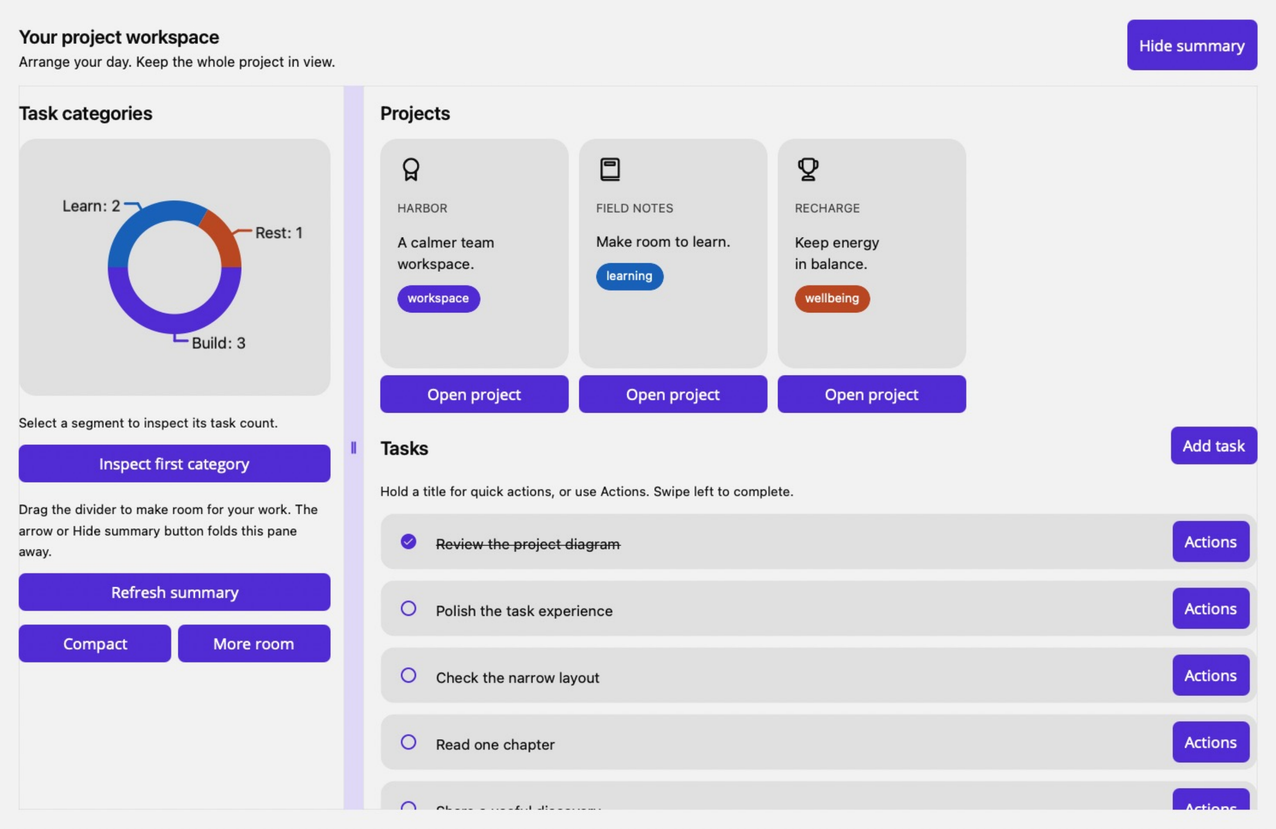
TOOLKIT: ONE-TIME SETUP

NuGet: Syncfusion.Maui.Toolkit 1.0.11

```
using Syncfusion.Maui.Toolkit.Hosting;
builder.ConfigureSyncfusionToolkit();
```

MauiProgram.cs, after creating the builder and before builder.Build().

Improve one project workspace



Arrange

Give each part of the work enough space.

Inspect

Look closely without leaving the project.

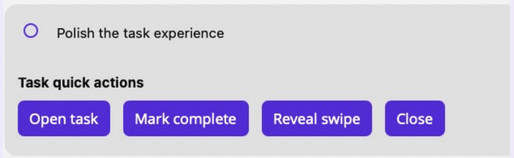
Act

Use clear, deliberate task actions.

Make task actions clear

Page BindingContext: a task view model with ShowActionsCommand and CompleteCommand.

Hold or choose Actions



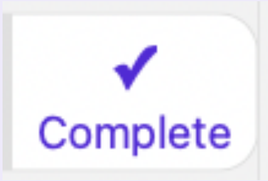

```
<Label Text="Hold for task actions">
  <Label.GestureRecognizers>
    <LongPressGestureRecognizer
      MinimumPressDuration="600"
      Command="{Binding ShowActionsCommand}" />
  </Label.GestureRecognizers>
</Label>
```

Attach the recognizer to the target view.

Keep a visible button or menu as an alternative.

MinimumPressDuration is in ms. Android uses its system duration, not this value.

Set swipe action colors

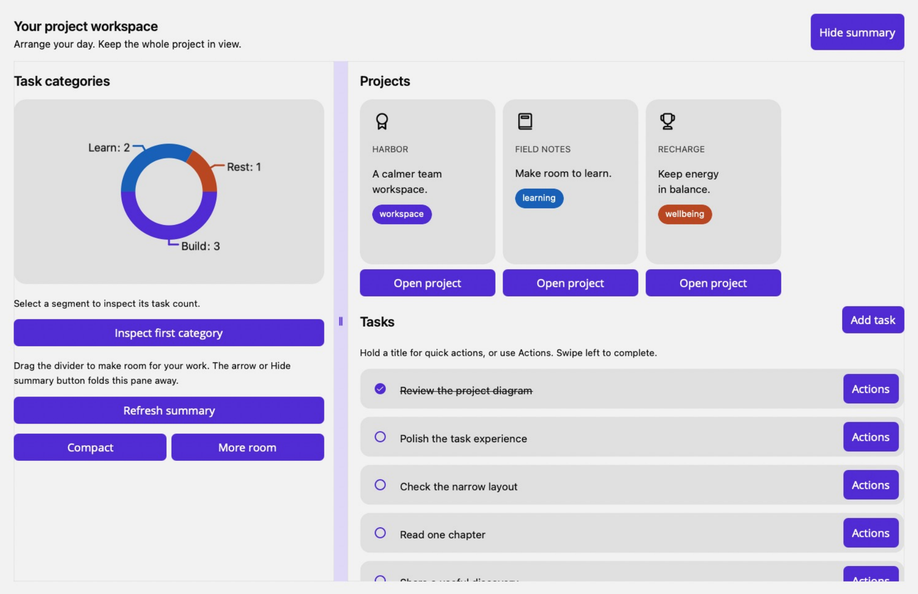


Keep the icon and text readable in both app themes.

```
<SwipeView.RightItems>
  <SwipeItems>
    <SwipeItem Text="Complete"
      IconImageSource="check.png"
      Command="{Binding CompleteCommand}"
      IconColor="{AppThemeBinding
        Light=#502AD4, Dark=White}"
      TextColor="{AppThemeBinding
        Light=#502AD4, Dark=White}" />
  </SwipeItems>
</SwipeView.RightItems>
```

Give the workspace adjustable panes

PAGE ROOT: xmlns:toolkit="http://schemas.syncfusion.com/maui/toolkit"




```
<toolkit:SfGridSplitter
  Orientation="Horizontal" HeightRequest="360">
  <toolkit:SplitterPane x:Name="ProjectPane"
    Size="240" MinimumSize="160"
    IsCollapsible="True">
    <Label Text="Projects" />
  </toolkit:SplitterPane>
  <toolkit:SplitterPane Size="*" MinimumSize="240">
    <Label Text="Project details" />
  </toolkit:SplitterPane>
</toolkit:SfGridSplitter>
```

Size: fixed units or * sizing, not percentages.

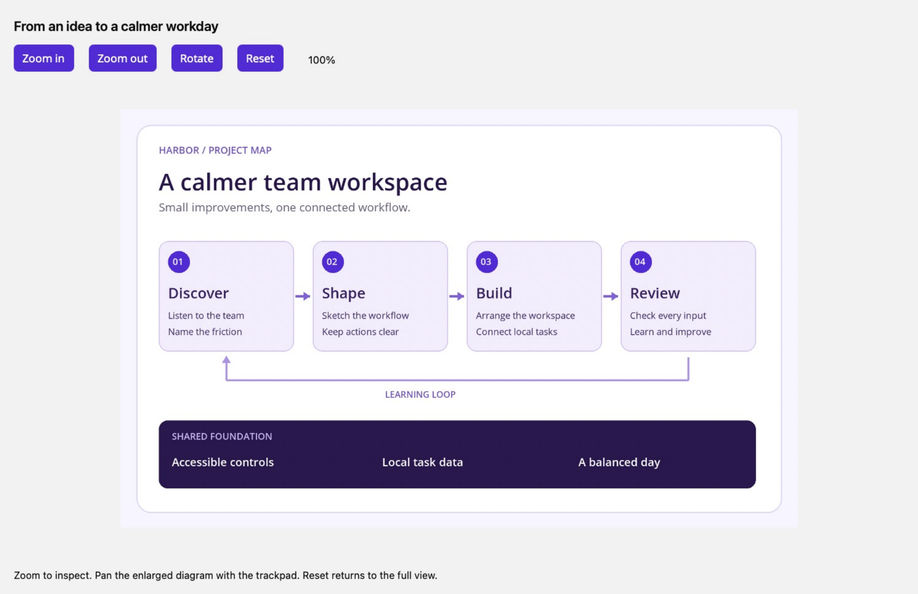
MinimumSize sets a limit. Keep the parent bounded.

Optional code-behind toggle

```
ProjectPane.IsCollapsed =
  !ProjectPane.IsCollapsed;
```

Inspect a project diagram in place

PAGE ROOT: xmlns:toolkit="http://schemas.syncfusion.com/maui/toolkit"




```
<toolkit:SfInteractiveViewer
  x:Name="ProjectViewer"
  MinimumZoomFactor="1"
  MaximumZoomFactor="4"
  HeightRequest="360">
  <Image Source="project.png" Aspect="AspectFit" />
</toolkit:SfInteractiveViewer>
```

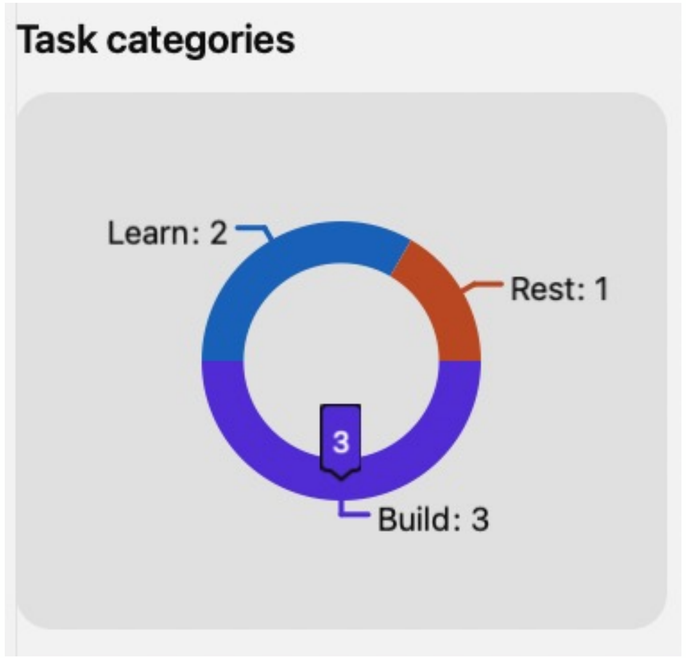
Add project.png as a MauiImage.

The viewer hosts the view; the image is its content.

```
ProjectViewer.Rotate();
ProjectViewer.Reset();
```

Improve the chart already in the app

PAGE ROOT: xmlns:toolkit="http://schemas.syncfusion.com/maui/toolkit"




```
<toolkit:SfCircularChart>
  <toolkit:SfCircularChart.TooltipBehavior>
    <toolkit:ChartTooltipBehavior
      UseSeriesFillColor="True"
      Stroke="White" StrokeWidth="2" />
  </toolkit:SfCircularChart.TooltipBehavior>
  <toolkit:DoughnutSeries
    ItemsSource="{Binding Costs}"
    XBindingPath="Name" YBindingPath="Value"
    EnableTooltip="True" />
</toolkit:SfCircularChart>
```

Page BindingContext: Costs contains items with Name (string) and Value (double).

EnableTooltip belongs on the series.

A border needs both Stroke and StrokeWidth.

Demo: one project, one app

01

Arrange

Resize the dashboard
Inspect a category

02

Inspect

Open the project diagram
Zoom, pan, reset

03

Act

Use task actions
Change the theme

One app switch. No setup on stage.

Put a badge on the right control

NEW IN RC1: TABBEDPAGE

```
<TabbedPage>
  <ContentPage x:Name="InboxPage"
    Title="Inbox"
    TabbedPage.BadgeText="3"
    TabbedPage.BadgeColor="Red"
    TabbedPage.BadgeTextColor="White" />
</TabbedPage>
```

Attached properties go on the direct child page.

SHELL: EXISTING BADGE API

```
<Tab Title="Inbox" BadgeText="3"
     BadgeColor="Red" BadgeTextColor="White">
  <ShellContent
    ContentTemplate="{DataTemplate vm:InboxPage}" />
</Tab>
```

Keep Shell tabs in a Shell app.

vm:InboxPage is your page type.

```
TabbedPage.SetBadgeText(InboxPage, "4");
TabbedPage.SetBadgeText(InboxPage, null);
```

Update or bind the text. null hides it; "" shows a dot. Apple may retain system badge colors.

Make navigation intent explicit

1 REGISTER | APPSHELL CONSTRUCTOR

```
Routing.RegisterRoute(
  "trip/{tripId}", typeof(TripPage));
```

2 NAVIGATE | CALLER

```
await Shell.Current.GoToAsync(
  "//routes/trip/SEA-204");
```

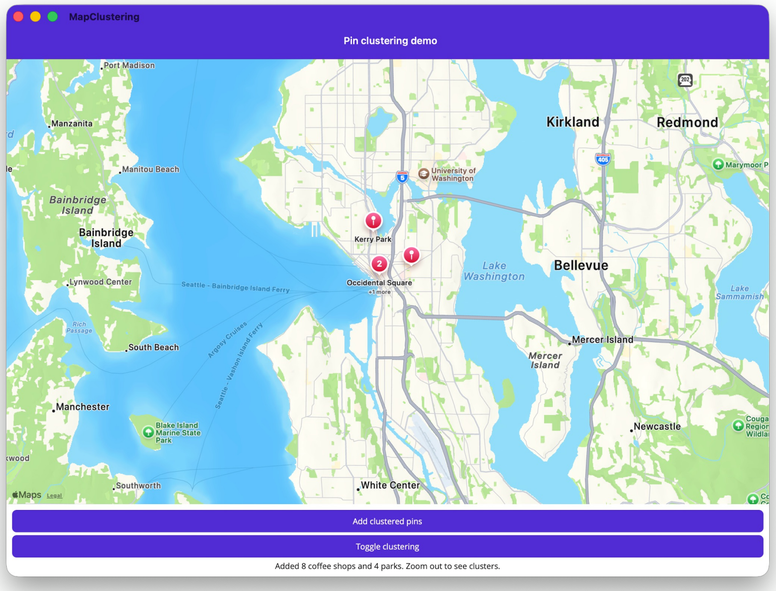
The example Shell has a ShellContent with Route="routes". Templates use an absolute route.

3 RECEIVE | DESTINATION PAGE

```
public class TripPage : ContentPage, IQueryAttributable
{
    public void ApplyQueryAttributes(IDictionary<string, object> query)
    {
        Title = (string)query["tripId"];
    }
}
```

Make a dense map readable

PAGE ROOT: xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps"




```
<maps:Map IsClusteringEnabled="True"
          Region="47.60,-122.33,0.04,0.04">
  <maps:Map.Pins>
    <maps:Pin Label="Office"
      Location="47.600,-122.330"
      ImageSource="office_pin.png" />
    <maps:Pin Label="Meeting"
      Location="47.601,-122.331" />
  </maps:Map.Pins>
</maps:Map>
```

NuGet: Microsoft.Maui.Controls.Maps

`MauiProgram.cs: builder.UseMauiMaps();`

Visual: official Mac Catalyst sample, Preview 7 snapshot.

Map keys: Android = Google Maps; Windows = Azure Maps; Apple = no map key.

Clustering: Android, iOS, Mac Catalyst. Location permission is only for location features.

Keep sign-in native

1 App

Get options from the server.

2 Platform

Run the native credential prompt.

3 Server

Verify and create the session.

using Microsoft.Maui.Authentication; | optionsJson comes from your server

```
if (!Passkeys.IsSupported)
    throw new PlatformNotSupportedException();

var asserted = await Passkeys.AssertAsync(optionsJson);
var responseJson = asserted.ToString();
```

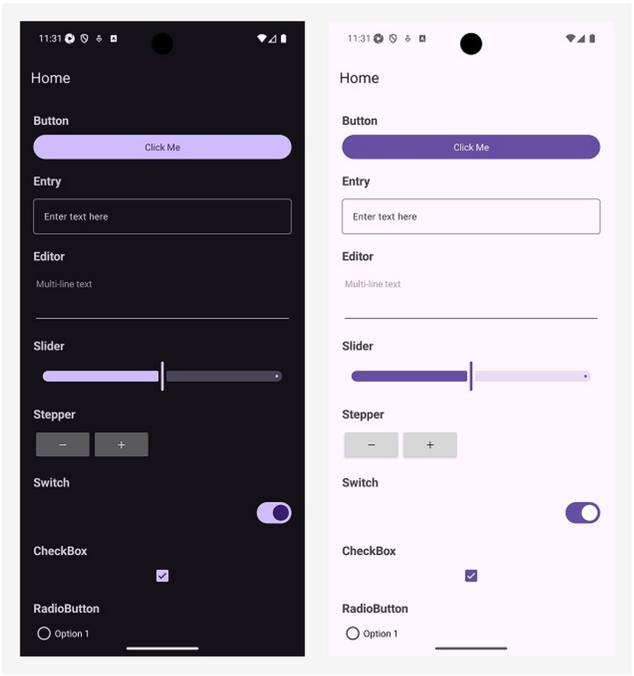
Send responseJson as raw application/json to the server. Do not encode it as a JSON string.

Register: CreateAsync | Sign in: AssertAsync | Support check: IsSupported

Setup: HTTPS domain + Apple Associated Domains / Android Digital Asset Links.

Enable Material 3 on Android

ANDROID | OPT-IN



Project file: .csproj

```
<PropertyGroup>
  <UseMaterial3>true</UseMaterial3>
</PropertyGroup>
```

Build and run the Android target.

The default is false.

New in .NET 11:

Material helper types are public for handler customization.

Official Android control examples. Material 3 is not a Mac Catalyst theme.

Match launch assets and system bars

Themed splash assets

.csproj / ItemGroup

```
<MauiSplashScreen
  Include="Resources/Splash/splash.svg"
  Color="#FFFFFF"
  DarkFile="Resources/Splash/splash-dark.svg"
  DarkColor="#000000"
  DarkTintColor="#FFFFFF"
  BaseSize="128,128" />
```

Supply both SVG files.

Keep one MauiSplashScreen item.

No dark metadata: existing behavior.

System bars: two separate controls

Backgrounds: Android opt-in in PropertyGroup

```
<MauiAndroidSystemBarsUseMauiChrome>
  true
</MauiAndroidSystemBarsUseMauiChrome>
```

Icons: the app Window, on Android and iOS

```
window.StatusBarTheme =
    StatusBarTheme.Light;
```

Light = dark icons on a light background.

Default follows the app theme.

Compile XAML. Load resources on demand.



SourceGen is the .NET 11 default in Debug and Release. No opt-in is needed.

Page resources and a consumer

```
<ContentPage.Resources>
  <RoundRectangle x:Key="CardShape"
    x:Shared="False" CornerRadius="16" />
</ContentPage.Resources>
<Border
  StrokeShape="{StaticResource CardShape}" />
```

Lazy loading is automatic for eligible entries.

x:Shared="False" means a new instance per lookup.

Template type + ancestor command

```
<DataTemplate x:DataType="vm:TaskItem">
  <Button Text="{Binding Title}"
    Command="{Binding
      Source={RelativeSource
        AncestorType={x:Type vm:TasksVm}},
      Path=AddTaskCommand}" />
</DataTemplate>
```

Set BindingContext to TasksVm; vm is its namespace.

x:DataType supplies a type, not the data object.

Lazy resources reduce page creation work

Automatic with SourceGen. Register factories now; create eligible resources on first use.

Reported PR benchmark | One page with 63 resources | Lower is better

Inflator	Time (microseconds)	Allocated (KB)
XamlC	336.88	352.97
SourceGen, no lazy	217.30	258.12
SourceGen + lazy	43.17	60.14

About 80% less time and 77% fewer allocated bytes

Versus SourceGen without lazy resources. Not whole-app startup; later access still has a cost.

Use C# expressions for small view logic

PROJECT PROPERTYGROUP

```
<EnablePreviewFeatures>true</EnablePreviewFeatures>
```

```
<ContentPage x:Class="Demo.CounterPage"
  xmlns:vm="clr-namespace:Demo"
  x:DataType="vm:CounterVm">
  <VerticalStackLayout>
    <Label Text="{$'Count: {Count}'}" />
    <Button Text="Add" IsEnabled="{!IsBusy}"
      Clicked="{(s, e) => AddOne()}" />
  </VerticalStackLayout>
</ContentPage>
```

Interpolate text. Negate a flag. Call a page method.

Keep larger logic in commands or code-behind.

CounterPage.xaml.cs

```
readonly CounterVm _model = new();

public CounterPage()
{
    InitializeComponent();
    BindingContext = _model;
}

void AddOne() => _model.Count++;
```

CounterVm notifies changes to Count and IsBusy.

Define AddOne once. Async lambdas are not supported.

SourceGen required. XAML Incremental Hot Reload: separate preview; default in Debug, with a Hot Reload host.

Upgrade behavior, not only the target

Navigation and lifecycle

Push, pop, tabs, flyouts and modals.

Check new Apple handler defaults.

Controls and input

Selection, scrolling, gestures and keyboard.

Check accessibility and explicit swipe colors.

Dependencies and custom code

Pin package versions. Review custom handlers.

Check dynamic code and reflection paths.

Published app

Exercise real state and data in Release.

Resolve trimming and NativeAOT warnings.

RC1: FlyoutPage uses a handler on Apple by default. Custom renderers need explicit registration.

More features in the terminal for you and your agents

dotnet run

Interactive device selection

dotnet watch

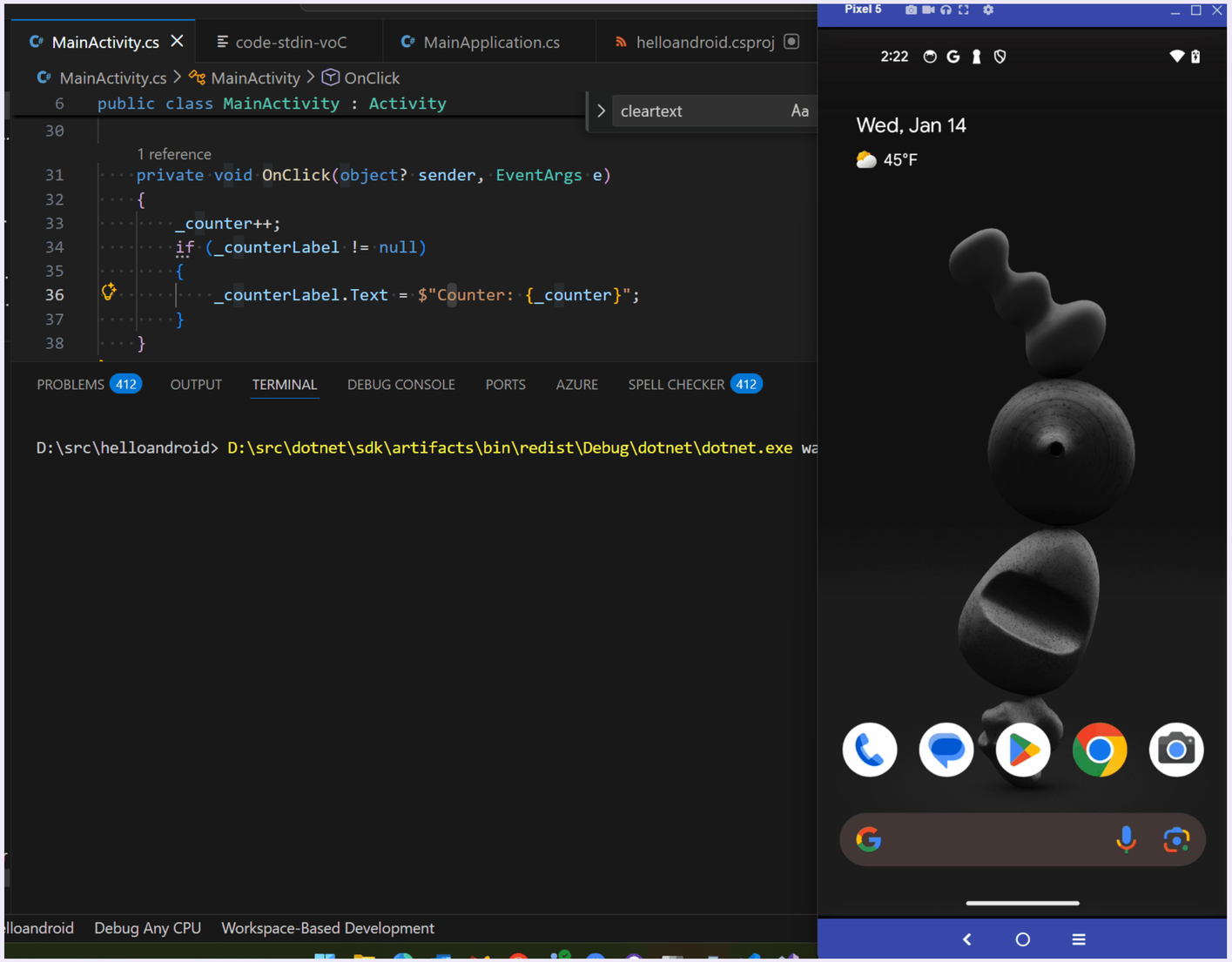
No debugger involved

*Blazor Hybrid (js/html/images)**

dotnet test

Microsoft Testing Platform

Android / iOS / MacCatalyst



RESOURCES

Try one improvement in your app

MAUI 11	learn.microsoft.com/dotnet/maui/whats-new/dotnet-11
Toolkit 1.0.11	github.com/syncfusion/maui-toolkit/releases/tag/v1.0.11
DeveloperBalance	github.com/dotnet/maui-samples > 10.0/Apps/DeveloperBalance

DeveloperBalance link: original .NET 10 sample. Feature-specific .NET 11 samples are in notes.

Questions

What would you improve in your app?

MAUI 11	learn.microsoft.com/dotnet/maui/whats-new/dotnet-11
Toolkit 1.0.11	github.com/syncfusion/maui-toolkit/releases/tag/v1.0.11
DeveloperBalance	github.com/dotnet/maui-samples > 10.0/Apps/DeveloperBalance